

Convergence Is Not Correctness

The sequential-specification campaign: independent, kernel-checked
specifications
for every replicated data type in Sal

Sal / FP Launchpad (KC + Claude)

2026-07-16; rewritten 2026-07-17

Abstract

Replication-aware linearizability certifies that every version of a replicated data type is the fold of some delivery-respecting linearization of its events. The reference point of that guarantee is the datatype’s *own* fold: a bug in `do` appears identically on every replica, converges perfectly, and passes every convergence theorem. This note records the campaign that closes the gap. For every production RDT in Sal there is now a kernel-checked theorem that the fold of any single-replica history equals the output of a straightforward sequential program, the one a programmer would write with no replication in mind. The campaign’s unifying observation is the *witness discipline*: sequential interfaces address state positionally (`dequeue()` is nullary, `insert` names an index), replicated operations cannot, and every replicated operation therefore carries the identity of what its issuer *observed* the positional referent to be; the honesty side conditions of the theorems are exactly the faithfulness of those witnesses. The campaign’s negative results carry as much information as its theorems: the rehoming tombstone-free RGA is refuted at this specification on a four-op single-replica trace, its fused Peritext inherits the refutation at the render, and Shesha passes sequentially while failing at the merge, so *do*-faithfulness and merge-faithfulness are independent axes, and both separations are kernel-checked. A final section states what the campaign does *not* give: per-replica soundness and convergence do not compose into “the merged state is some user’s sequential result,” and the precise corner where composition fails is the dead-anchor corner the sequence-CRDT trilemma names. Every claim below names its Lean identifier.

1 The specification gap

The framework’s central guarantee is RA-linearizability at every reachable configuration: each version the store registers is $\text{fold}(\rho)$ for some enumeration ρ of the version’s event set that respects delivery order. This is the right guarantee for *replication*: the merge invented nothing, reordered nothing it was not entitled to reorder, and all replicas agree.

It says nothing about whether the fold is *right*. The reference sequence is the datatype’s own `do`; if `do` is wrong, every replica is wrong in unison (Figure 1). Two incidents made this concrete in this repository. First, a mark-extent defect in an early Peritext render passed every convergence obligation and was caught only by eye. Second, and decisively: the rehoming tombstone-free RGA carries a fully general, kernel-clean RA-linearizability capstone (`rga_ra_linearizable3_eq`), and its `do` nonetheless reorders surviving text when an interior element is deleted, with no concurrency anywhere in the history (§6). Both defects are invisible to convergence because convergence is self-referential.

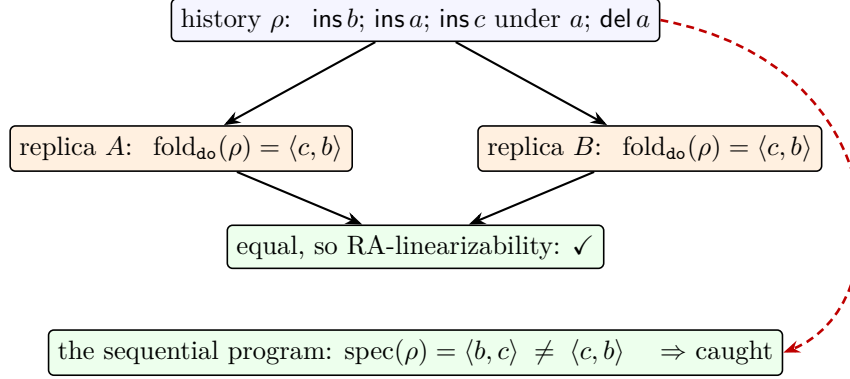


Figure 1: Why convergence cannot see a `do` bug. A defective fold is applied identically everywhere, so all replicas agree and every convergence theorem passes over the shared mistake. Only a reference computed by something *other than* `do` (dashed) can disagree. The history shown is the real refutation trace of §6.

The remedy is an independent specification. For sequential behaviour there is a canonical one: *the sequential program a programmer would write for the same operations with no replication in mind*. A counter is addition. A queue is a list. A text buffer is insert-at-position and delete. A marked-text editor is a buffer of characters and boundary tokens with one left-to-right walk.

2 The obligation, and the rules of engagement

The campaign’s obligation, per datatype:

$$\text{for every sequentially honest single-replica history } \rho: \\ \text{read}(\text{fold}(\rho)) = \text{spec}(\rho)$$

proved by discovering an inductive invariant of the fold, never by weakening the spec to match the implementation. “Sequentially honest” is the datatype’s own issuing discipline specialized to a linear history (fresh, monotone stamps; each operation applicable at the state it executes against), so the theorems attack no strawman: the histories quantified over are exactly the ones a single well-behaved replica produces, and in every case the side condition reuses a contract the convergence proofs already consume.

Three method rules were fixed in advance and held throughout. *Falsifiable statement first*: each tier states its equality before proof work begins, and a refutation is a first-class outcome (three landed, §6). *The spec is external*: the sequential program is written from the datatype’s user-facing intent, never read off the implementation; where the two disagree, the disagreement is the result. *Tests carry both polarities*: every concrete test block pairs its expected values (hand-derived, never evaluated out of the code under test) with should-fail companions, so the harness demonstrably discriminates; the refutation files pair each negative with the positive prefix on which the two sides still agree, isolating the departing operation exactly.

3 The observation gap: positional interfaces, witnessed ops

Writing out the specifications exposes one structural pattern, and it is the campaign’s most instructive artifact. A sequential structure addresses its state *positionally or implicitly*: `dequeue()`

takes no argument because “the head” is unambiguous; `insert(i, e)` names an index; `write(v)` implicitly overwrites everything before it. A replicated datatype can afford none of this, because positions and implicit referents are not stable under concurrent edits: by the time an operation is applied elsewhere, “the head” may be a different element and index i a different character. So the replicated operation carries a **witness**: the identity (tag, stamp, id, coordinate, snapshot) of what the issuing replica *observed* the positional referent to be, at its own state, at issue time (Figure 2).

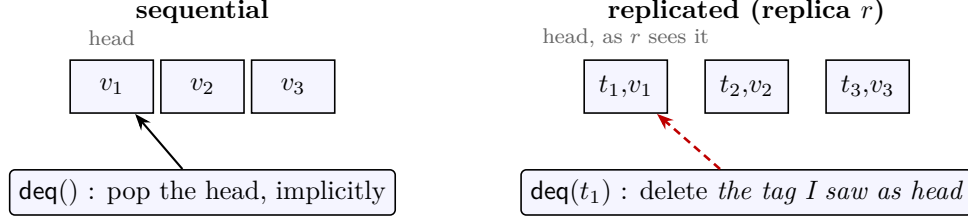


Figure 2: The observation gap, on the queue. The sequential `dequeue` is nullary; the replicated one names the tag its issuer read as the head of its own replica (dashed: the witness). Honesty is the faithfulness of the witness: `qOK` states that the fold at the issue point is literally $(t_1, v_1) :: \text{rest}$.

The sequential-honesty side conditions of §2 are then exactly the statement *the witness is what the sequential operation would have seen*, and each campaign theorem says: under that discipline, the witnessed-identity program implements the positional program. Where no witness is needed the alphabets coincide and the spec is a view homomorphism. The whole catalogue of §4 sorts by this principle, and the summary is one sentence: *every honesty condition in the campaign is the faithfulness of a witness that replaced a positional argument*.

4 The specifications, datatype by datatype

4.1 No gap: the alphabets already coincide

For nine of the flat datatypes the RDT operation is literally the sequential operation; the state carries invisible bookkeeping (tags, per-replica slots) and the spec is the evident program on the user-visible view.

- **Counter, increment-only counter.** Spec state $\mathbb{Z}$ (resp. $\mathbb{N}$); increment is $n \mapsto n + 1$. The fold *is* the history length (`counter_seq_sound`, `ioc_seq_sound`).
- **PN-counter.** Spec: `#inc - #dec` (`pnSpec`); `pn_seq_sound`.
- **Grow-only set and map; conditioned G-set.** Spec: accumulation; x is a member iff some operation added x (`goset_seq_sound`, `gomap_seq_sound`, `gsetcond_seq_sound`).
- **OR-set, plain and efficient.** Spec state: a plain set; `add e` inserts, `rem e` deletes (`orSpecStep`). The tag machinery (fresh (t, e) stakes; remove filters every tag of e) is invisible to one replica: the element view of the fold is the naive program (`orset_seq_sound`, `orsete_seq_sound`), a fold homomorphism, no invariant needed.
- **Enable-wins flag.** Spec state: one boolean; `enable` $\mapsto$ true, `disable` $\mapsto$ false (`ewSpecStep`); `ewflag_seq_sound`.

- **Add-wins priority queue.** A two-part spec pins both state components: membership behaves as the plain add/remove set with inc inert (`awpqSpecStep`, `awpq_mem_seq_sound`), and the increment component is a grow-only log of exactly the issued increments (`awpq_inc_log_sound`).

4.2 Gap: arbitration by stamp instead of program order

LWW and FWW registers. The sequential register is write v with *program order* deciding who wins; the spec programs are “the last write’s value” (`lwwSpecFold`) and “the first write’s value” (`fwwSpecFold`). The RDT operation carries its Lamport stamp *as the arbitration key in the payload* (a max- resp. min-semilattice on stamped writes), because concurrent replicas have no shared program order to appeal to. The gap is a transfer of ordering authority from control flow to data, and the honesty condition is precisely its repair: **stampsMono** (stamps strictly increase along the history, the sequential Lamport condition), under which the arbitration key agrees with program order and `lww_seq_sound` / `fww_seq_sound` recover last- and first-write semantics. Without **stampsMono** the statements are false, and should be: a replica that back-dates a write is asking LWW to disagree with program order.

4.3 Gap: overwrite by snapshot instead of “everything”

Multi-valued register. The sequential write v implicitly overwrites everything before it. The RDT operation is `write( $v, O$ )` where O is a **prepare-time snapshot**: the write-stamps the issuer currently sees, named explicitly so that a *concurrent* write (absent from O) survives, which is the point of an MVR. The sequential spec is still the plain last-write register (`mvrSpecFold`). Honesty (`mvrOK`): the stamp is fresh and O lists *exactly* the issuer’s visible stamps, the formal form of “I overwrite precisely what I can see.” Under it, `mvr_seq_sound`: the visible-value view of the fold is the last write. The proof needs the campaign’s first genuinely discovered invariant, `mvr_over_tags`: every overwritten stamp was staked, which is what makes a fresh stamp provably not-yet-overwritten.

4.4 Gap: dequeue() becomes delete-what-I-saw

The **mergeable queue**, the cleanest specimen (Figure 2). The sequential queue is

$$\text{enq } v : S \mapsto S \uparrow [v] \qquad \text{deq}() : S \mapsto \text{tail}(S).$$

The RDT operation is `deq  $t$` : remove *by tag*, t being the tag of the element the issuing replica read as the head of its own queue; the implementation filters t out of the whole list, wherever it sits. The spec keeps the sequential shape, `tail`, blind to t (`qSpecStep`). Honesty (`qOK`) is witness faithfulness: the fold at the issue point is literally $(t, v) :: \text{rest}$. The invariant `q_tags_nodup` (fresh stakes stay duplicate-free) makes filter-by-tag remove exactly the head, and `queue_seq_sound` closes the loop: delete-what-I-saw implements pop-the-head. Its axiom audit is the leanest in the campaign: $\{\text{propext}, \text{Quot.sound}\}$, no choice anywhere. The same witness discipline is what the convergence side already consumed (`qHonest_of_applicable`); nothing was invented for the occasion.

4.5 Gap: insert-at-index becomes insert-after-who-I-saw

The embedded-chain RGA. The sequential text buffer is `insert( $i, e$ )` and `delete( $i$ )`, index-addressed. The RDT operation is `ins( $e, \pi, a$ )`: a is the *id* of the element the issuer saw at position

$i - 1$ (its anchor; the sentinel 0 for the head), and π is the anchor’s stored coordinate as the issuer read it (the proof-carrying prefix); $\text{del}(x)$ names the id seen at position i . The spec program is the buffer in id-clothing (**eSpecStep**): front-cons for $a = 0$, otherwise splice immediately after a ’s entry (**eInsAfter**); delete filters by id. On a sequential history the anchor id determines the index uniquely, so this *is* the positional program with positions named by their occupants. Honesty (**eSeqOK**): stamps exceed all prior insert stamps, and each operation is applicable at the current fold, i.e. the anchor is live and carries exactly π . The theorem (**embed_seq_sound**, §5) is the strongest form in the campaign: the canonical sorted state equals the spec buffer *record for record*. The same spec, verbatim, is what the rehoming RGA is refuted against (§6); its gap is not the witness (the rehoming op carries one too, plus a path) but the implementation moving *other* elements’ referents on delete.

4.6 Gap: one mark becomes a boundary pair

Fused Peritext. The sequential marked-text editor has an *atomic* $\text{mark}(i, j, m)$. The fused RDT has no such operation: a mark is **two** ordinary inserts of boundary elements $\langle m \rangle$ and $\langle /m \rangle$ sharing a fresh mark id, anchored at the range ends; mark removal is two deletes. An atomic positional operation is traded for a *pair* of witnessed inserts, which is the atomicity horn of the mark-positioning trilemma, accepted knowingly. The spec therefore lives at the token level: the editor is the §4.5 buffer over $\text{char} \oplus$ boundary entries plus one left-to-right walk maintaining the open-mark set (a start boundary opens, the matching close removes, characters display with what is open). **peritextEmbed_seq_sound** says the datatype’s render is that editor’s screen, id for id in the **_ids** variant; the range-level reading is recovered by the positional intent theorems (**render_id_active_iff_between**: a character carries mark m iff it sits, in reading order, strictly between m ’s boundaries). The pair encoding is also why the character/boundary distinction is load-bearing in the delete theorem: deleting a *character* re-formats nothing (**renderIds_del**), while deleting a *boundary* is mark removal and must re-format its span (machine-checked necessary: **boundary_delete_breaks_filter_eq**).

5 The proofs: four tiers, three kinds of invariant

The campaign ran easiest to hardest, and the difficulty gradient turned out to be a gradient of invariant *kinds*.

Tier 1 (eleven flat datatypes), SeqSpec_Flat.lean. Mostly view homomorphisms, no invariant at all. The two exceptions need *identity hygiene*: facts about tag bookkeeping that hold on every reachable single-replica state and are exactly what the induction consumes. For the MVR, **mvr_over_tags** (overwritten stamps are staked); for the AWPQ, duplicate-freedom of stakes.

Tier 2 (the mergeable queue), MergeableQueue_SeqSpec.lean. One identity-hygiene invariant, **q_tags_nodup**, as described in §4.4.

Tier 3 (the embedded-chain RGA), EmbedRGA_SeqSpec.lean. The invariant becomes *geometric*. The spec splices an insert immediately after its anchor; the datatype sorts records by birth-chain coordinates; the bridge is the **adjacency lemma** (**chainBefore_snoc_iff**, Figure 3): a fresh insert’s chain $c_a ++ [\delta]$ sits directly after its anchor’s chain c_a in display order, because the fresh timestamp gap δ exceeds every delta any existing chain hangs off c_a , a fact that costs only

sum-telescoping (a chain’s deltas sum to its id, and every existing id is below the fresh stamp). The sorted canonical state then *is* the spec buffer, record for record (`embed_seq_sound`). Two transports come for free. The published tombstoned RGA is read-equivalent to the embed on honest event sets (the compaction theorem), so it inherits the buffer spec: `rga_seq_read_eq_buffer` and element-level `rga_seq_read_element` (`EmbedRGA_SeqSpec_RGA.lean`, a standalone build target because the two RGA models share top-level names). And the sided generalization simulates the one-sided instance on all-R histories unconditionally (`sFold_lift0p`), so the buffer spec transports to the sided datatype’s all-R fragment at zero proof cost; the full two-sided theorem has since landed as `sided_seq_sound` (`SidedRGA_SeqSpec.lean`): a sentinel-rooted buffer where an R-insert splices immediately after its anchor and an L-insert immediately before it, via the two sided adjacency lemmas (`schainBefore_snocR_iff`, `snocL_iff`, axioms `{propext, Quot.sound}`).

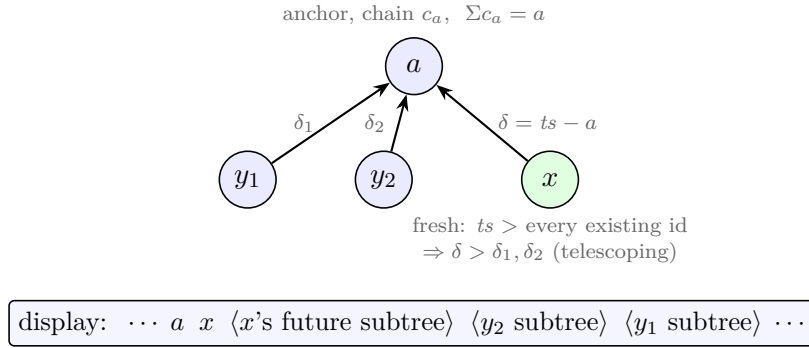


Figure 3: The adjacency lemma, the geometric heart of tier 3. Siblings display newest-first; a fresh insert’s delta beats every existing child’s because deltas telescope to ids and every existing id is below the fresh stamp. Hence the newcomer lands *immediately* after its anchor: exactly what the sequential buffer’s splice does.

Tier 4 (fused Peritext), `PeritextEmbed_SeqSpec.lean`. The invariant kind is *composition*, and that is the finding: tier 3’s buffer soundness, made payload-generic, already says the fold’s (id,element) sequence is the naive buffer, and the datatype’s render and the editor’s screen are literally the same pure fold over that sequence (`peritextEmbed_seq_sound`). No new invariant was needed. This is also why the tier had to wait for the re-based Peritext: against the rehoming kernel the same statement is false (§6).

6 The refutations

The statement shape is falsifiable per datatype, and three refutations landed, all machine-checked, each with its honesty guards discharged on the witness so the failure is never an artifact of a malformed trace.

The rehoming RGA fails at do (`rehoming_seq_refuted`). Figure 4. Four operations on one replica, honest in the datatype’s own terms (every op `accurate` and `fresh_ts` at its state): build $\langle b, a, c \rangle$ with c under a , then delete a . The naive buffer keeps survivor order, $\langle b, c \rangle$; the rehoming do re-homes c to the root, where the newest-first sibling tiebreak leapfrogs it over b : the read is $\langle c, b \rangle$. The should-pass half (`seq_agrees_before_delete`) shows both sides agree on the delete-free prefix, isolating the delete as the exact point of departure. The design’s convergence capstone is

untouched: it converges, to its own reordering fold. This refutation is what demoted the design from the repository’s canonical seat.

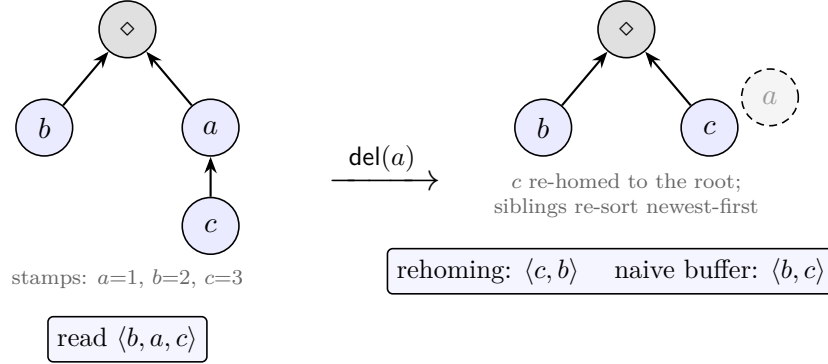


Figure 4: The four-op refutation. One replica, no merge. Deleting a swaps the surviving b and c in the rehoming RGA’s read; the naive buffer keeps their order. The embedded-chain RGA reads $\langle b, c \rangle$ on the same trace: c ’s immutable coordinate still carries a ’s chain as pure geometry, so nothing moves.

Fused Peritext on the rehoming kernel fails at the render (fused_delete_reformats_survivor). The same reorder lifted to char $\oplus$ boundary: deleting a plain character re-homes a survivor past a mark boundary, and an untouched character changes formatting, on one replica. This upgraded the re-base of Peritext onto the embed kernel from housekeeping to a correctness fix. On the embed kernel the corresponding *general* theorem holds: **renderIds_del** (axioms {propext, Quot.sound}): deleting a character leaves every other character’s rendered formatting bitwise untouched, because character entries are open-set-neutral and delete is a filter on an order-canonical state.

Shesha passes sequentially and fails at the merge. The mirror image. Shesha’s single-replica reads are the naive linked list (**sequential_soundness**, kernel-clean), and it uniquely preserves survivor order under delete among the tombstone-free designs of its generation (**delete_preserves_survivor_order**). Its *merge* is machine-checked non-RA-linearizable (**Shesha_Rows_Refuted**): at an honest reachable configuration the three-way merge splits a deleted node’s children around a concurrent sibling, producing a read that is the fold of no delivery-respecting linearization.

7 The two-axis separation

Together the refutations witness that do-faithfulness and merge-faithfulness are *independent* axes (Figure 5): the rehoming family occupies merge-yes/do-no, Shesha do-yes/merge-no, and the embedded-chain family shows the top corner is achievable, by one design idea in both sequence datatypes: immutable birth coordinates, so that deletion moves nothing and the merge decides nothing.

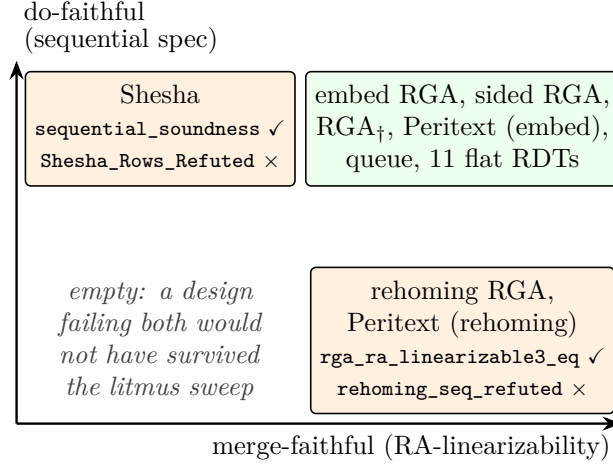


Figure 5: Every placement is a named kernel theorem. The two failure modes are orthogonal: converging to a reordering fold (bottom right) and folding correctly into a non-linearizable merge (top left).

8 What composition does not give

It is tempting to compose the guarantees: RA-linearizability says every version is $\text{fold}(\rho)$ for some delivery-respecting ρ over the event union; the campaign says folds of sequentially honest histories equal the spec program. May one conclude that every merged document is the naive program’s output on *some* ordering of everyone’s operations? **No**, and the failure is precise.

The campaign’s theorems quantify over *sequentially honest* histories; an RA-linearizability witness is delivery-respecting but need not be sequentially honest on either count. Stamp monotonicity fails harmlessly: when the state is a function of the event set (fold-canonicity, `e_fold_canon`), any enumeration can be re-sorted. Applicability is the real corner (Figure 6): in the union of two branches, an insert’s anchor may be deleted by a *concurrent* operation carrying a smaller Lamport stamp, so in stamp-sorted order the delete precedes the insert, the anchor is dead at the insert’s prefix, and the naive buffer no-ops the splice, while the datatype (whose coordinates travel in the op) places the element correctly. Some interleavings of the union match the spec program and some do not.

Whether a matching order always exists is genuinely open, and we record it as such:

Open Question 1 (oq:seqwitness). *On every honestly reachable configuration of the embedded-chain RGA, does the event union admit a delivery-respecting enumeration that is also sequentially honest, so that the merged read is the naive buffer’s output on it? Caution from adjacent territory: for the rehoming design the analogous “order dependency edges first” discipline is refuted (the DepComp refutation, via rehoming non-transitivity). The embed’s immutable coordinates remove that refutation’s mechanism, but no proof exists in either direction.*

Independently of the question, the composition that *is* sound and kernel-checked today: every version is the datatype’s fold of a delivery-respecting enumeration (RA-linearizability); the fold is canonical per event set (`e_fold_canon`); each replica’s own contribution obeyed the sequential spec while it was being made (this campaign); and merge-level intent is delivered by separate, named theorems where it holds (delete-order preservation as sublist stability, subtree convexity as non-interleaving, read-equivalence to the published RGA as the compaction theorem). The campaign

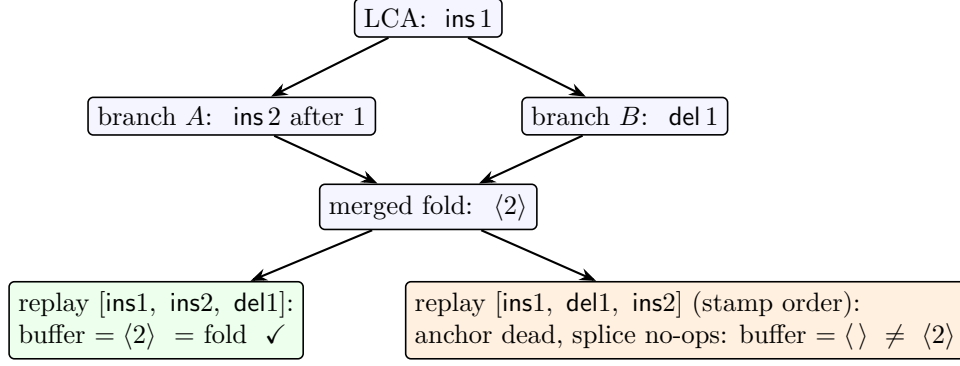


Figure 6: The dead-anchor corner. The merged fold keeps 2 (OR-set survival on immutable coordinates); whether a naive-buffer replay of the union agrees depends on which delivery-respecting order is chosen, and the stamp-sorted one can be wrong. This is the same corner the sequence-CRDT trilemma names for insert-after-already-deleted-anchor.

does not manufacture a merge-level user story; it pins the single-replica story so that merge-level claims have something sound to stand on.

9 The verified matrix

The anomaly-comparison matrix (task #64, [whiteboard/anomaly-matrix/](#)) ran six implementations and four production libraries through 25,917 merges per row, every cell backed by executed code. Its in-repo rows have since been upgraded from machine-run to kernel-checked; the table below is the matrix’s load-bearing core with each cell a Lean identifier. External libraries (Yjs, Automerge, Loro, list-positions) remain empirical by nature; their rows live in the matrix report unchanged.

design	sequential spec (do)	RA-lin (merge)
embedded-chain RGA	<code>embed_seq_sound</code>	<code>embed_ra_linearizable3</code>
sided RGA (two-sided)	<code>sided_seq_sound</code>	<code>sided_embed_ra_linearizable3</code>
published RGA _†	<code>rga_seq_read_eq_buffer</code>	<code>rgaWithTombstones_ra_lin..._eq</code>
Peritext (embed)	<code>peritextEmbed_seq_sound</code>	<code>peritextEmbed_ra_linearizable3</code>
mergeable queue	<code>queue_seq_sound</code>	<code>queue_ra_linearizable3</code>
eleven flat RDts	<code>*_seq_sound</code>	<code>*_ra_linearizable3_eq</code>
rehoming RGA	<code>refuted: rehoming_seq_refuted</code>	<code>rga_ra_linearizable3_eq</code>
Peritext (rehoming)	<code>refuted (render): fused_delete_reformats_surv...</code>	<code>peritext_ra_lin..._eq</code>
Shesha	<code>sequential_soundness</code>	<code>refuted: Shesha_Rows_Refuted</code>

Every positive theorem above is kernel-clean (axioms $\subseteq \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$; the queue’s and `renderIds_del`’s audits omit choice). Refutations by concrete witness follow the SPOT convention (`native_decide`, adding the compiler axioms); the refuted *statements* are quantified properties whose honesty guards are discharged on the witness.

10 Mechanization map

tier / row	file (relative to Sal/ConditionedMRDTs/)
tier 1: eleven flat RDTs	MRDT_Instances/SeqSpec_Flat.lean
tier 2: mergeable queue	MRDT_Instances/MergeableQueue/MergeableQueue_SeqSpec.lean
tier 3: embedded-chain RGA	MRDT_Instances/EmbedRGA/EmbedRGA_SeqSpec.lean
tier 3, two-sided: sided RGA	MRDT_Instances/SidedRGA/SidedRGA_SeqSpec.lean
tier 3 corollary: $\text{RGA}_\dagger$	MRDT_Instances/EmbedRGA/EmbedRGA_SeqSpec_RGA.lean (standalone)
tier 4: fused Peritext	MRDT_Instances/Peritext_Embed/PeritextEmbed_SeqSpec.lean
render fix theorem	MRDT_Instances/Peritext_Embed/PeritextEmbed.lean
negative: rehoming RGA	MRDT_Instances/RGA_Rehoming/RGA_SeqSpec_Refuted.lean
negative: rehoming Peritext	MRDT_Instances/Peritext_Rehoming/Peritext_Read.lean §10
negative: Shesha merge	MRDT_Instances/Shesha/Shesha_Rows_Refuted.lean